

Swedbank AI



★ Member-only story

Open in app ↗



Medium



Search



Write



Banking Needs

Predicting customer finances using deep learning on time-series



Philippe de Meulenaer, PhD · 9 min read · Apr 3, 2020



Time series prediction has been a ubiquitous data science area of interest for decades, with many proposed methods each with successful applications for different types of time series.

1. Introduction

In this post, we are going to present a use case within banking where deep learning (DL) turns out to be an efficient solution for time series prediction. The value of correct predictions of customers' account balances is crucial in modern banking and has many different applications such as services providing guidance towards a more sustainable economy with advice tailor-made for each customer.

While this is an actual application, synthetic data has been used for the purpose of this post. Also, some simplifications have been made in order to provide an easy-to-grasp overview. Several sequences of one years' worth of synthetic transactional data has been created with the objective to predict cash flows 3 months into the future, as illustrated in the figure below.

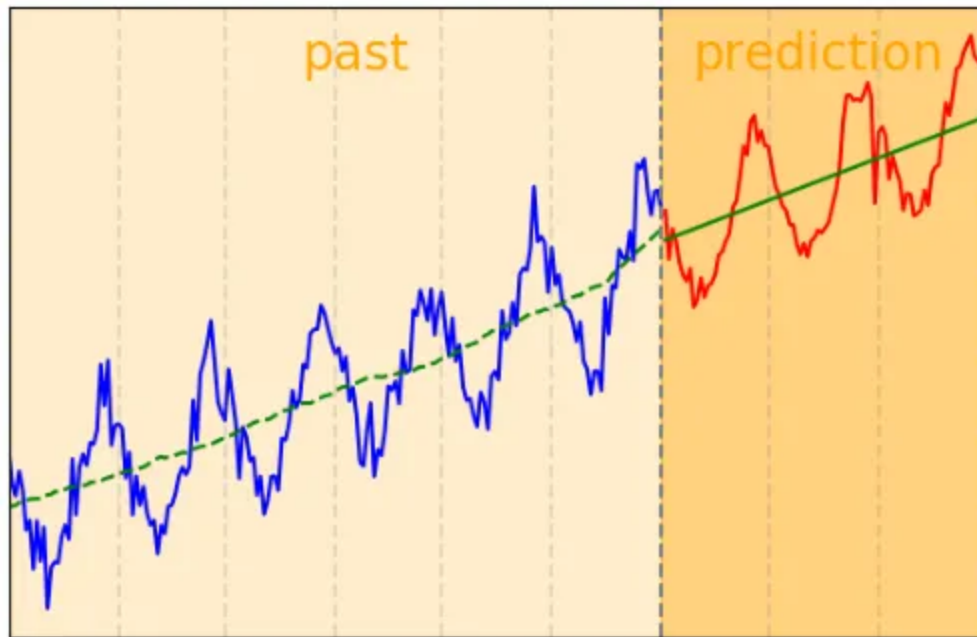


Fig. 1. Illustration of the objective: forecasting customer cash flows.

Technological stack reviewed in this article:

- Deep learning (1-D CNN) for time series prediction
- Petastorm for Spark-TensorFlow interaction
- Pandas_UDF for model serving

2. The data: generation of synthetic time series

In our real use case, the observation pool contains several million time series, one for each individual customer at a given period. The time series consists of transactions, aggregated over each day, across all relevant accounts.

In order to mimic different types of time series we've encountered, the following types of patterns were artificially generated: sinus functions, triangles, squares, and sawtooth patterns (see examples in Fig. 2). Gaussian noise was added to introduce a degree of randomness for each account's

transaction history. For the long-term trend, we selected an exponential component to make the problem non-trivial. We also introduced several sharp increases or decreases to simulate unusually large money transfers, as such features are a challenging aspect of classical time series forecasting methods.

We created 2 data sets: one containing 100K time series to be used for training the model, the other containing 2.5M time series, to represent the “unknown” data set on which the model has to be served. The generated time series are 15 months long, with the first year being used as historical data, and the last 3 months what we will predict.

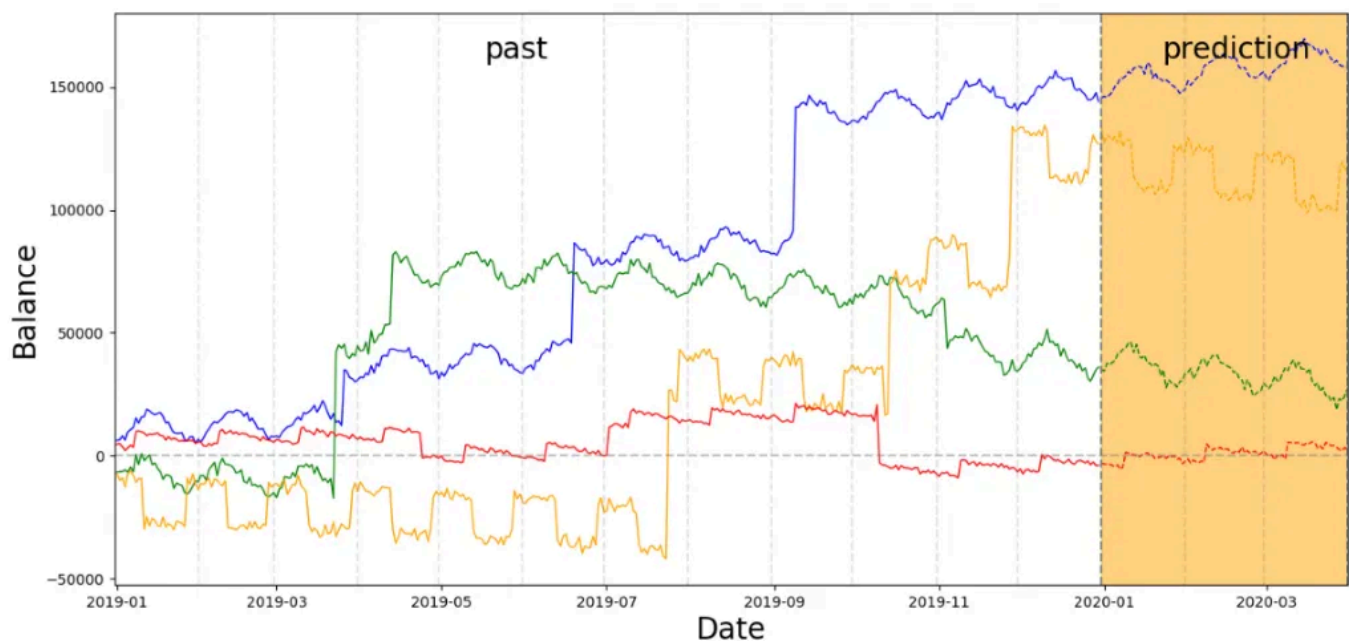


Fig. 2 Example of synthetic time series, with 4 different signal types. The objective will be to use the left 1-year part (white background) and predict the right 3-months part (yellow background).

3. Prediction with “Prophet”

To benchmark our deep learning efforts we’ve also used the well-known Prophet package, authored by Facebook to predict time series with few sharp

variations. Notably, ad-hoc filtering such as sharp variations can improve the predictions.

Prophet is a regression technique where the goal is to fit a long-term trend along with some seasonality:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t.$$

where

- $g(t)$ is the trend function that models usually long term and non-periodic changes in the value of the time series,
- $s(t)$ represents periodic changes, e.g. weekly, monthly and yearly seasonality, and
- $h(t)$ stands for holidays which might irregularly occur over one or more days.

Seasonality was adapted to monthly, and the holiday feature was not used.

The technique was applied on time series displaying one or more sharp variations. A second model was also fitted on the same time series with the sharp variations filtered out (and added back after the fit).

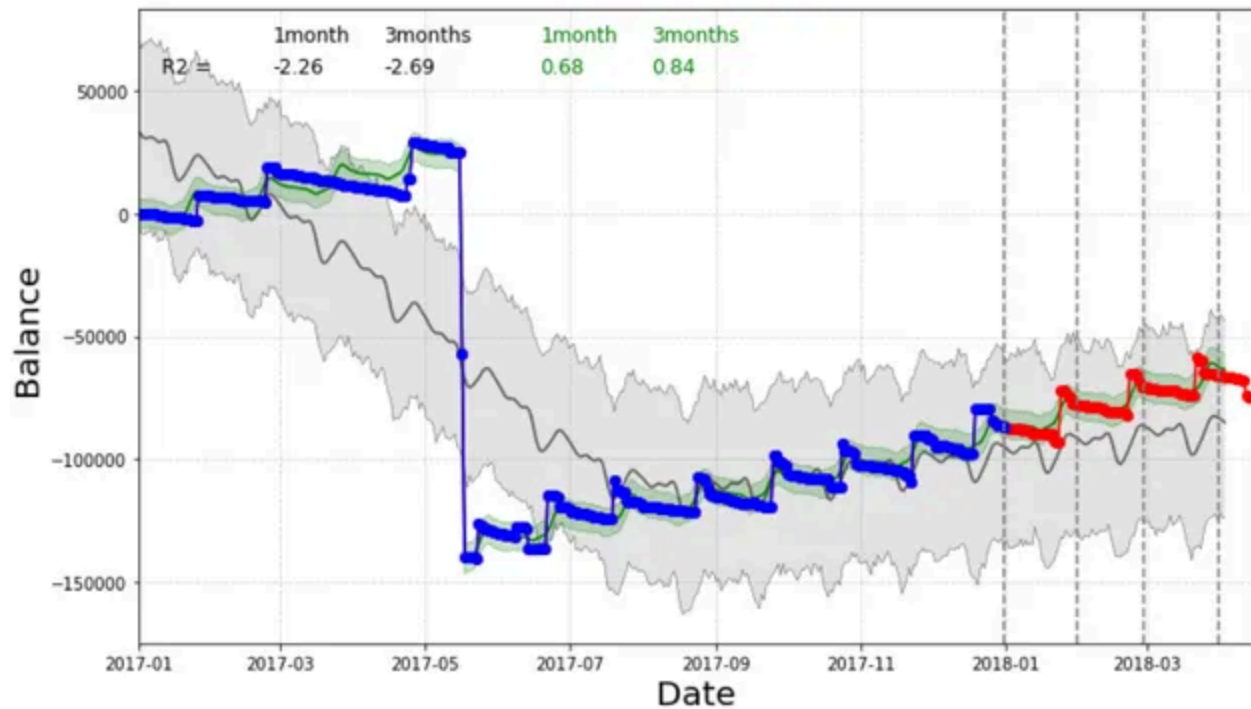


Fig. 3. Comparison of the application on Prophet regression models on a time series with a sharp variation.

In the figure above, we show the comparison of the first model (in black, with the uncertainty area in gray) and the corrected model (model 2, in green, with associated green uncertainty area). Historical data is represented by the blue time series, and future true data by the red one. In most cases, we found that the second choice, where sharp variations were filtered, lead to more accurate fits — confirmed by visual inspection as well as goodness-of-fit-metrics.

In real-life situations, distinguishing sharp anomalous variations from normal variations is far from trivial. Moreover, another drawback is that this methodology only allows for the prediction of univariate time series and not multivariate ones.

In conclusion, although Prophet can be very useful in many cases, we decided to explore another, more powerful and scalable prediction method using deep learning.

4. The deep learning solution

a. The solution architecture

The high-level scheme of the method we chose to develop is shown below:

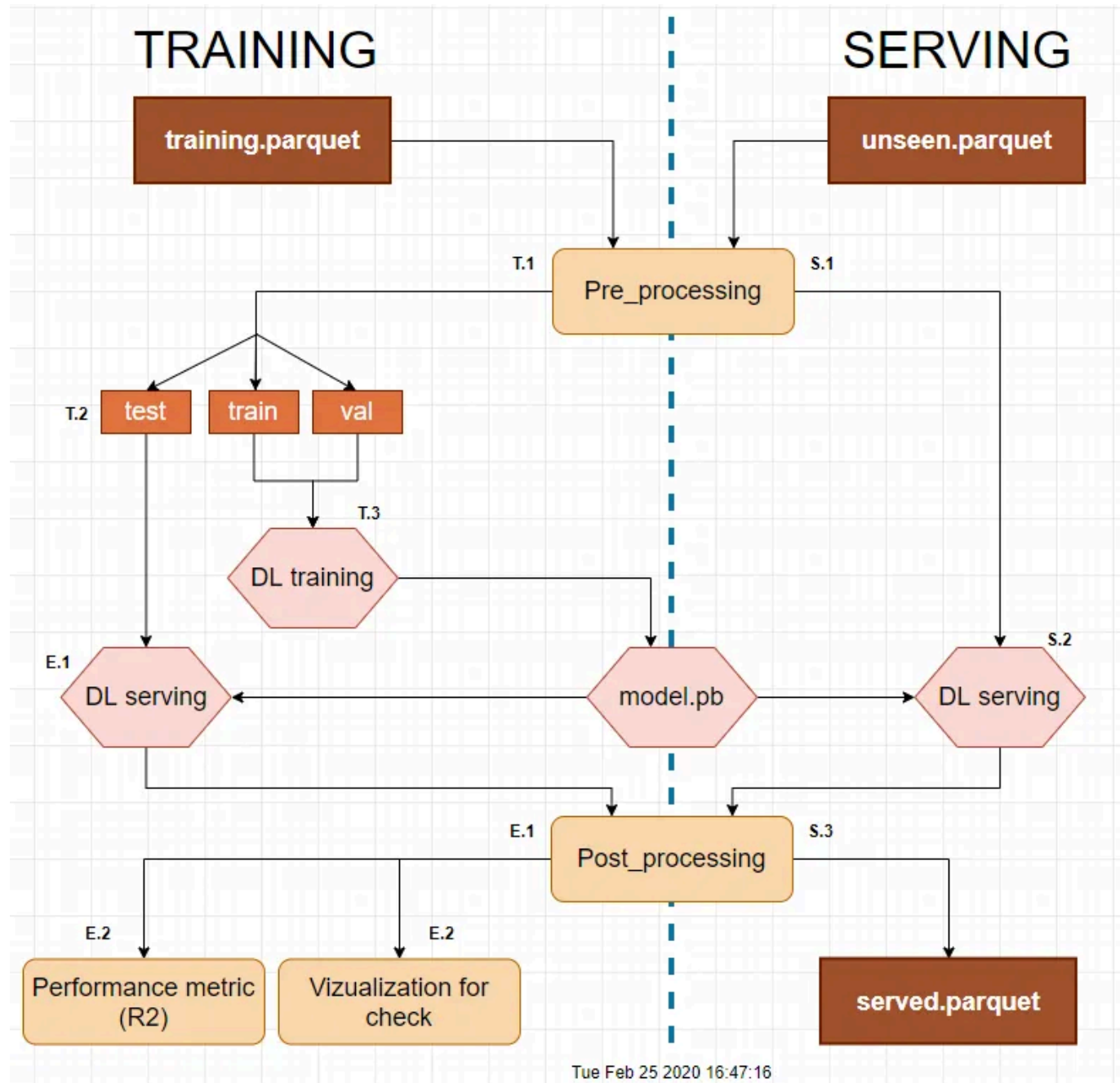


Fig. 4. Architecture of the deep learning solution.

On the training side (left part of the dashed blue line) a time series data set is used to produce the model. It first undergoes a pre-processing step where

the time series are scaled — i.e. mean subtracted and divided by the standard deviation as this makes the deep learning solution more robust. In the following step, the data set is split in three parts, 1) *train* and 2) *validation* for model building, and 3) *test* for performance evaluation of the final model.

In the serving stage (right-hand side of the diagram), the model can be served on unseen data. Then the model prediction takes place, and finally the time series including the predictions are scaled back within the post-processing step.

b. The CNN model

The selected deep learning model is a simple one-dimensional Convolutional Neural Network (CNN). Although Recurrent Neural Networks, like the well-known LSTM could seem like the natural choice for sequence prediction tasks, we found that 1-D CNNs give comparable prediction performance on this task. The key is that training CNNs is ~10x faster using GPUs whereas LSTMs do not allow for an equally efficient GPU usage scheme in our case. Numerous examples of such implementation of CNNs on time series prediction tasks can be found — for example [here](#), [here](#), or [here](#).

Conversely to previous approach, where one regression model was fitted per time series, here the strategy is to fit one 1-D CNN model for the whole batch of 100K time series. The advantages of this approach are:

- It is computationally much cheaper than a one-model per time series approach.
- The model “learns” from many time series at once.

The model is defined as a simple *tf.keras* model:

```
def define_1dcnn_model(N_days_X, N_days_y, model_conf):
    '''
    :param N_days_X: Number of days for training
    :param N_days_y: Number of days to be predicted
    :param (dict) model_conf: dictionary of model hyperparameters
    :return (keras model): A compiled 1DCNN keras model
    '''

    hyperparameters = model_conf['hyperParameters']

    opt = tf.compat.v1.train.AdamOptimizer()

    # tf.keras, Functional API
    inputs = tf.keras.layers.Input(
        shape=(N_days_X, 1,), name='input')
    x = tf.keras.layers.Conv1D(
        filters=int(hyperparameters['filters']),
        kernel_size=int(hyperparameters['kernel_size']),
        activation=hyperparameters['activation'])(inputs)
    x = tf.keras.layers.MaxPooling1D(
        pool_size=int(hyperparameters['pool_size']))(x)
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(
        int(hyperparameters['dense_units']),
        activation=hyperparameters['activation'])(x)
    outputs = tf.keras.layers.Dense(
        N_days_y, name='output')(x)
    model = tf.keras.models.Model(inputs=inputs,
                                   outputs=outputs)

    # Model compilation
    model.compile(optimizer=opt,
                  loss=hyperparameters['loss'])

    return model
```

which afterwards is converted to a TensorFlow estimator (this, because the solution was designed before TF2.0):

```
estimator = tf.keras.estimator.model_to_estimator(
    keras_model=model,
    model_dir=model_dir,
    config=run_config)
```

Once the TensorFlow model is defined, our focus shifts to finding the best way to feed large data sets for its training.

c. Petastorm, or how to feed big data into TensorFlow

Classically, TensorFlow accepts inputs in the form of Numpy arrays as well as Datasets (built-in TensorFlow format). Until the end of 2018, feeding data from Spark to TensorFlow Datasets was handled using TFRecords, a somewhat cumbersome TensorFlow file format. Then came Petastorm, a library released by Uber which enables TensorFlow to directly access parquet files for the model training, using the *make_batch_reader* method, as shown below.

```
with make_batch_reader(input_files_train,
                       num_epochs=num_epochs) as train_reader:

    with make_batch_reader(input_files_val,
                           num_epochs=1) as val_reader:

        def input_fn(reader, batch_size, time_steps_X,
                     time_steps_y, feature_dim):
            dataset = transform_reader(reader, batch_size)
            dataset = dataset.prefetch(batch_size)
            return dataset

        train_spec = tf.estimator.TrainSpec(input_fn=lambda:
            input_fn(train_reader,
                    batch_size,
                    time_steps_X,
                    time_steps_y,
                    feature_dim=1),
            max_steps=max_steps)

        eval_spec = tf.estimator.EvalSpec(input_fn=lambda:
            input_fn(val_reader,
                    batch_size,
                    time_steps_X,
                    time_steps_y,
                    feature_dim=1),
            steps=None)

        tf.estimator.train_and_evaluate(estimator, train_spec,
            eval_spec)
```

In the above, the *transform_reader* function reshapes the input and converts the data read by the Petastorm reader into TF.Dataset format. It also makes sure that the data is transferred in the defined sized and shuffled batches:

```
def transform_reader(reader, batch_size):  
    def transform_input(x):  
        inputs = tf.reshape(x.X, [-1,N_days_X,1])  
        outputs = tf.reshape(x.y, [-1,N_days_y])  
        return (inputs, outputs)  
  
    return make_petastorm_dataset(reader).map(transform_input)\  
        .apply(unbatch()).shuffle(3*batch_size, seed=42)\  
        .batch(batch_size, drop_remainder=True)
```

More info on Petastorm can be found on the [Uber Github repo](#). There is also a simple and great example available on [Azure Databricks](#).

Then, when the model is trained, it can be serialized in a ProtoBuffer file (TensorFlow model file format) using the *export_saved_model* command:

```
# Saving the estimator to .pb file  
def serving_input_receiver_fn():  
    inputs = {  
        model.input_names[0]:  
            tf.placeholder(tf.float32, [None,N_days_X, 1]),  
    }  
    return tf.estimator.export.ServingInputReceiver(inputs, inputs)  
  
export_dir = estimator.export_saved_model(model_dir+'/model',  
    serving_input_receiver_fn)
```

We followed this strategy to train the model on 100K time series as this size is sufficient to obtain satisfactory performance for real-world data.

Now the model is trained, and its artefact can be used to serve the full bank of 2.5M time series.

d. The (distributed) serving strategy: the Pandas_UDF trick

Again, as the data size is relatively large, Spark is the tool of choice for serving the model. The serving requires loading the TensorFlow model and apply it on each time series. To do this, we made use of the Pandas_UDF tool, introduced since Spark 2.3.

The serving function of the model is a Python function, not a native Spark one, which means that we need to distribute it manually, i.e. to have the function distributed on each row of the data frame containing the time series.

The classical way of distributing a Python function in Spark is by using User Defined Functions: UDFs. The problem is that the execution process might be very slow if the Python function is not optimized. In our specific case the Python function needs to deserialize the TensorFlow model and apply it on the 365 days array to predict the next 92 days. This means that for each row of the 2.5 million rows long data frame, the model needs to be re-loaded. This is clearly a non-efficient distributed scheme.

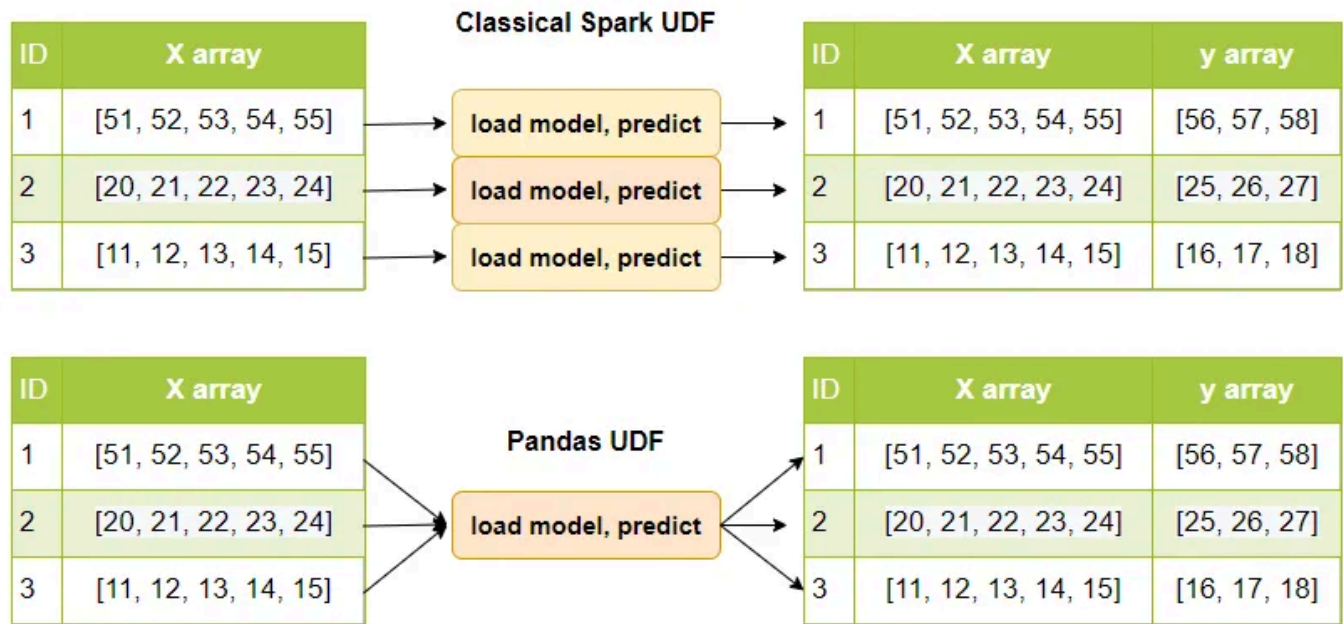


Fig. 5. The Pandas UDF speed-up vs classical Spark UDF.

Contrary to classical Spark UDFs, the main idea behind Pandas_UDF is to vectorize the serving Python function using a library called Pyarrow. This turns the input and output into arrays (or *Series*) rather than single rows. This means only one de-serialization of the model per Spark executor, as illustrated in Fig. 5.

Here is the code snippet for the classical UDF case:

```
@F.udf("array<float>")
def udf_scoring(x):
    predictor_fn = tf.contrib.predictor.from_saved_model(export_dir =
export_dir_saved)
    return np.around(predictor_fn({'input': np.array(x).reshape(-1,
N_days_X, 1) })['output'][0].tolist(), decimals=3).tolist()

ts_balance = ts_balance.withColumn('y_pred', udf_scoring('X'))
```

In contrast, the Pandas_UDF snippet looks like:

```
@F.pandas_udf("array<float>")
def pandas_udf_scoring(x):
    predictor_fn = tf.contrib.predictor.from_saved_model(export_dir =
export_dir_saved)
    return Series([np.around(predictor_fn({'input':
np.array(v).reshape(-1, N_days_X, 1)})['output'])[0], decimals=3) for
v in x])

ts_balance = ts_balance.withColumn('y_pred', pandas_udf_scoring('X'))
```

We compared the run of classical Spark UDF with Pandas_UDF and found that the decreased computational expenditure was approximately in the order of 60. This is not surprising since the single-row distribution scheme of classical UDFs hits a bottleneck. The TensorFlow model must be re-loaded at every call of the function, which makes it a time-demanding task.

e. The prediction & conclusion

A few examples of time series predictions are given in following picture.

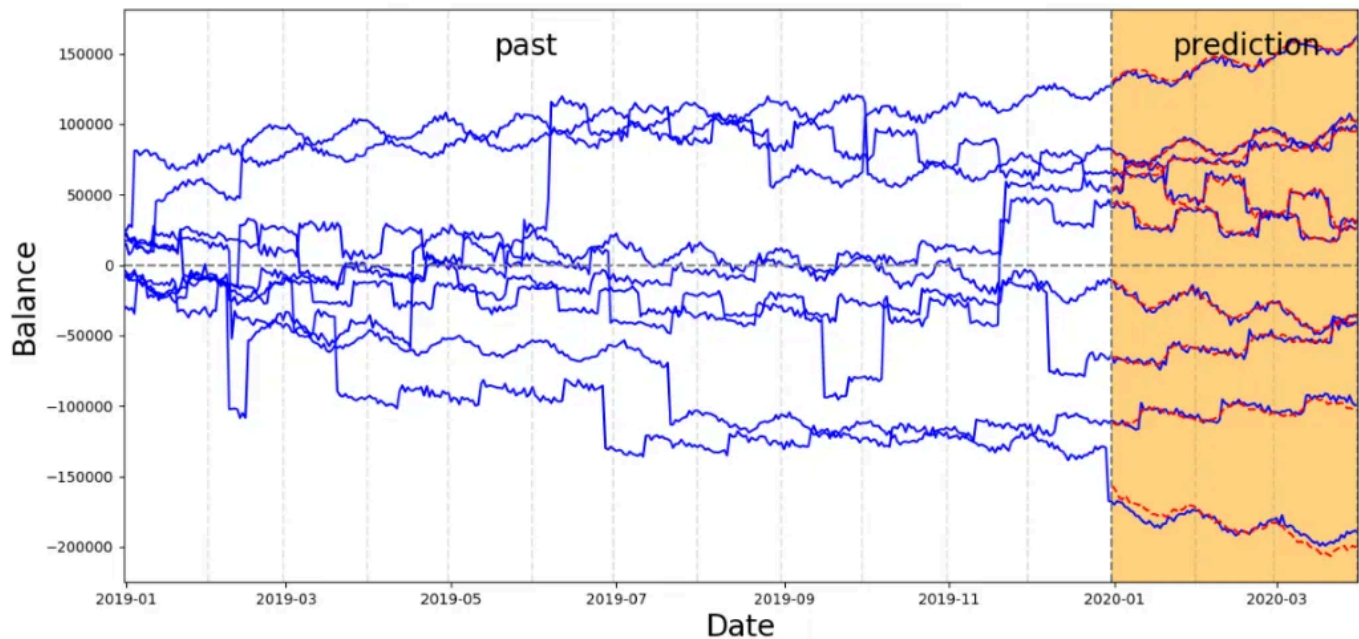


Fig. 6. Prediction for a few time series (in red) compared to the “truth” (dashed blue), in the prediction time window.

This work demonstrates the ability of creating a deep learning solution for a “big data” time series prediction task on a distributed environment. 1-D CNN is confirmed to be a well-tailored time series modelling technique, **Petastorm** an efficient library that allows the model to crunch the data on the training side, while **Spark Pandas_UDF** is the tool of choice to distribute the model when serving it on unknown data.

The above serves to illustrate an example of one of many applications within the **Analytics & AI** team at **Swedbank**. It uses Spark, is adapted for large amounts of data, and adapted to our development platform, Databricks cluster within Azure cloud.

However, this code could easily be adapted to a small data sample (for example training and serving on a few thousands time series data sets) for simple testing on a laptop. The code behind the deep learning solution will be made available very soon after this post publication.



Written by Philippe de Meulenaer, PhD

255 followers · 1.3K following

Follow

I am working as a freelance consultant MLOps engineer. I am very interested in cloud technologies for Machine Learning deployments



Published in Swedbank AI

74 followers · Last published Jan 15, 2021

Following

AI, machine learning and deep learning at one of the largest banks in the Nordics.

